Pillowfort v4-CS1: Wire Contract and Failure Semantics

Protocol v4 · Profile CS1 · 9 November 2026

This appendix specifies the consent-based succession profile of protocol v4. It separates required wire checks, endpoint persistence, relay decisions, and recovery outcomes. The evidence bundle contains the exact source, executable M1–M4 models, configurations, counterexamples for deliberately weakened models, implementation tests, and browser measurements. None is an independent security audit or an unbounded proof.

Tested implementation: `10fe8f8b490c9e890d5a642bc25568e985c2fe8a`. The claim-to-test matrix, bounded analysis report, and complete evidence bundle identify the checked scope.

A. Profile and interoperability

Profile identifier: `pf-v4-cs1` (9 November 2026). The transport version remains `4`; the profile identifier is a specification label, not a negotiated wire field. All participants and the relay must implement this profile. The integer `4` alone does not establish mixed-version compatibility.

Contract	Earlier host-available design	v4-CS1
Planned transfer	Signed offer and target acceptance; outgoing host validates	Unchanged
Host disappears	No disconnected-host delegation in that design	Signed join-order policy permits offers after disconnect grace
Recipient consent	Explicit acceptance	Explicit acceptance or refusal, including after host loss
Public application metadata	Ordinary application content remains inside MLS	Only signed succession policy/response metadata may appear in `authorityEvent`
Strict-schema compatibility	Does not recognize the added event variants and sidecar fields	Requires matching endpoints and relay; no legacy fallback

This is a wire-affecting profile extension, not an editorial reinterpretation of the old overview. Unknown-field rejection means older parsers cannot be assumed to interoperate. Use a coordinated deployment and recreate rooms when changing incompatible profiles. No claim is made that a profile label automatically negotiates that migration.

B. Encoding, identities, and signatures

IDs are canonical unpadded base64url encodings of 16-byte values. Public Ed25519 keys are 32 bytes; signatures are 64 bytes. A room instance is distinct from its human room code. An application event ID is distinct from the outer envelope message ID and from an authorization or grant token. Succession authorization IDs are derived as specified in F.2; they are not independently allocated random IDs.

B.1 Signed application event

The unsigned object has **exactly** these fields:

Field	Type and check
`v`	Integer `4`
`roomInstance`	16-byte room-instance ID; equals the receiving engine's room
`eventId`	16-byte unique event ID
`deviceId`	16-byte device ID; equals the authenticated MLS sender
`deviceSequence`	Positive safe integer; checked against the member's retained sequence
`logicalOrder`	Positive safe integer; equals the order attached to the accepted frame
`content`	Exactly one recognized, bounded content variant

A signed event adds only `signature`. Sign:

```
UTF8("Pillowfort Secure Room Application Event v4\0" + canonicalJson(unsignedEvent))
```

`\0` denotes one NUL byte, not the two printable characters backslash and zero. Canonical JSON sorts object keys, preserves array order, rejects negative zero/non-finite numbers, requires NFC strings and valid Unicode, and rejects unknown/non-data properties. The supported subset is bounded to depth 16 and 8,192 encoding nodes; application events are at most 60 KiB. Ed25519 verification disables ZIP-215 permissive behavior. A valid signature is necessary, not sufficient, for an authorized transition.

B.2 Invitation/member binding and resume

The invitation-authentication key signs UTF-8 JSON of this ordered array:

```
["pillowfort:secure-room-member-binding:v4", 4, 1,
 mode, roomId, roomInstance, deviceId, admissionId,
 signaturePublicKey, keyPackageDigest]
```

`mode` is `founder` or `admission`. `keyPackageDigest` is SHA-256 of the exact MLS KeyPackage bytes, encoded as canonical base64url. The immutable binding is checked against the actual KeyPackage and authenticated MLS credential; possessing an invitation is not host approval.

A retained device proves resume with its device signing key over:

```
["pillowfort:secure-device-resume:v4", 4, 1,
 roomId, roomInstance, deviceId, connectionId, requestId, challenge]
```

The server challenge is 32 random bytes. Invitation-key authentication cannot replace this proof for an established device.

C. Ordered frames and relay decisions

An envelope has exactly `v`, `suite`, `roomInstance`, `messageId`, `route`, `payload`, plus `to` only for device routing. `payload` is a complete MLS wire message, or the specified Welcome/tree admission bundle. The relay payload bound is 64 KiB; the complete WebSocket frame bound is 96 KiB.

A wire grant contains `v:4`, `suite:1`, `roomInstance`, `requestId`, `tokenId`, `deviceId`, `logicalOrder`, and `expiresAt`. The relay additionally binds the grant to the authenticated socket's **server-held `connectionId`**. That connection binding is not an extra wire-grant field. Only one current grant exists. The signed event commits to room/device/order, not to every grant field; the remaining grant checks are relay-state checks.

Before issuing an ordinary non-host grant, the relay also requires the current host to be connected with an empty delivery backlog. Otherwise it retains the exact queued request until a host delivery ACK or host resume completion makes preview possible. Issuing that grant early is unsafe for progress: a faster guest socket can submit before the host ACK arrives and receive `recipient-unavailable`. The exact preauthorized disconnection-succession candidate is exempt because its signed response deliberately does not require the offline host's preview.

For an ordinary application frame:

```
{kind:"relay", relayKind:"application", grant, envelope}
```

The relay validates socket ownership, active membership, empty sender delivery backlog, route, current grant, exact order, expiry, barriers, and replay fingerprint. An ordinary non-host frame is retained for host preview. Host software inspects a temporary MLS engine, verifies the event and legal transition, and signs no replacement message: it sends an exact `application-decision` referring to that pending frame. Relay acceptance is persisted before dependent effects; recipients independently validate and persist before acknowledging.

C.1 M1 transition contract

Starting condition / input	Guard	Durable mutation	Emission / recovery
Ready; local action	Active, resume complete, applicable barriers absent	Retain post-encryption MLS state, exact ciphertext, event, outbox, application rollback context	Send only after write; restart retries that operation, not regenerated ciphertext
Guest queued; host prefix pending	Current host must be connected and have acknowledged prior delivery	Retain exact request; no unusable grant	Host ACK/resume completion releases the queued grant
Ordered frame arrives	Exact room/device/socket/grant/order/expiry and route	Consume grant; retain pending frame or accepted result	Non-host ordinary frame goes to host preview
Host decision	Current host, exact pending message/order and unexpired approval	Store accepted/rejected outcome and replay fingerprint	Fan-out/result only after relay commit
Accepted result	Exact locally pending operation and order	Clear matching outbox/rollback; retain advanced MLS state	ACK after local write; release dependent work
Application rejection	Exact pending application	Restore application metadata; **do not restore old MLS sender state**	No sender-generation reuse
Outbound membership/update commit rejected	Exact pending commit	Retire local identity instead of restoring prior epoch	Fresh admission required; no epoch rollback
Duplicate frame/result	Same retained ID and fingerprint	Preserve original outcome	Retransmit/reconcile; never apply application twice
Ordinary delivery ACK	Exact retained backlog prefix	Remove only that prefix	Later or unknown IDs cannot erase unapplied entries

D. M2 admission transitions

State / input	Required binding or guard	Durable result and recovery obligation
Invitation-authenticated request	Exact room/device/admission/credential/KeyPackage binding; package digest unused	Pending record; not active and cannot device-resume
Host approves	Exact still-pending request and fingerprint; explicit caller authorization	Host may obtain Add grant; no Add before approval
Add committed	Current host; exact admission; supported one-member delta and update path	Admission enters `awaiting-welcome`; accepted Add creates cleanup obligation on cancellation
Welcome delivered	Exact admission and Add message; device-only route	Joiner persists MLS state and Welcome provenance
Bootstrap applied	Expected founder/host; exact admission and Welcome	History-redacted application snapshot; barrier remains
Profile proof submitted	Matching proof grant, device, Welcome, admission, MLS credential and signature	Retained proof pending host verification
Activation	Current host verifies exact proof and signing key	Active membership; complete admission barrier
Expiry/cancellation	Exact pending request at any phase	Never activate; post-Add cancellation schedules exact historical MLS removal
Authentication transmission/reload	A frame may have committed remotely	Retain `authentication-ambiguous` identity; reconcile the same artifact, never silently replace it

The first-party host approval window is 180 seconds; relay admission expires at 185 seconds from joining. Delay consumes the five-second completion margin. Expiry does not authorize reuse of the old approval, fingerprint, KeyPackage, or invitation-bound request.

E. M3 resume and peer disagreement

State / input	Guard and local obligation	Relay obligation
Retained device reconnects	Device-key proof binds current connection, request and challenge	Enter `pending` / `replaying-backlog`, not active
Replay entry	Verify original sender, exact metadata, MLS transition and replay identity; persist before proceeding	Retain backlog until exact replay completion
Historical Remove	Marker matches exact `(deviceId, admissionCommitMessageId)` and removal commit	Replay causal marker before its commit, even after original target retirement
Snapshot	Reconcile roster, signed host view, current transfer and removal evidence	Snapshot alone cannot create MLS membership or host authority
`backlog-end`	Snapshot reconciled; terminator matches the received replay boundary	End ID is final replay entry, or the owning resume request for an empty replay
`resume-complete`	All preceding replay work persisted; exact terminator	Atomically activate and clear replay backlog, then emit completion ACK
Active lifecycle repeated for an already admitted member	Verify roster/profile; with no matching admission barrier this is a no-op, including while an application is pending	This signal does not acknowledge or replace that application
Completion ACK	Exact locally outstanding completion	Ordinary outbound work may resume

Replay does not send separate ordinary `delivery-ack` frames for each replayed entry. The exact `resume-complete` terminator acknowledges that replay. Sending an independent prefix ACK first can change the relay's expected boundary; that is not the controller's protocol.

If one endpoint rejects an incoming commit, its engine restores its last valid committed snapshot and the controller fails that connection closed. It does not acknowledge the rejected delivery or continue ordinary sending. Other members may already have advanced. There is no automatic epoch merge, rollback of those peers, or quorum agreement protocol. Replaying a consistently invalid commit is not a repair mechanism. An irrecoverably divergent member needs fresh admission after retirement; loss of the necessary coordinator/state can require room termination. Outbound commit rejection and inbound validation failure are different cases and must not be conflated.

Storage failure during incoming commit processing likewise withholds acknowledgment and stops that receiver. A storage-failure experiment can establish isolation of that failed path, not agreement among arbitrary MLS forks. An unrelated active-lifecycle signal must not attempt an admission mutation: doing so would reject a returning device's durable application before its retry/result reconciliation.

F. M4 host transfer and public sidecars

F.1 Exact application content

```
{type:"host-succession-policy", deviceIds:[...]}
{type:"host-succession", action:"accept"|"reject", policyEventId, authorizationId}
```

Both are full signed application events under B.1 and are encrypted as ordinary MLS application payloads. The policy list must equal authenticated admission order, excluding the current host and pending-removal members. Display ordering and reconnect time do not change priority. The list is an internal implementation detail, not a displayed queue or setting.

The public policy sidecar is permitted only on `relayKind:"application"`. A response uses:

```
{kind:"relay", relayKind:"host-transfer-accept", grant,
 authorizationId, envelope, authorityEvent:signedResponse}
```

The historical transport variant name also carries a signed **refusal** in this profile; the signed `action`, not that variant name, determines whether authority changes. Arbitrary chat/profile/game plaintext cannot be put into `authorityEvent`.

Binding	Relay validation	Receiving endpoint validation
Signer	Sidecar key equals stored authenticated device credential	Inner signed device equals MLS sender and roster credential
Room and order	Sidecar room/order equal room and consumed grant	Inner room/order equal engine and delivered order
Policy	Current-host signature; eligible member IDs; retain policy event and outer frame IDs	Signed host policy and exact admitted order
Capability	Response's inner/outer authorization IDs equal current unexpired offer for that target	Decrypted response authorization/policy IDs equal delivered context
Retry	Outer canonical frame fingerprint includes sidecar, grant and ciphertext	Retained delivery digest rejects ID substitution
Consent	`action:"accept"` changes authority; `reject` advances offers	Apply signed choice, not delivery acknowledgment

The relay cannot decrypt MLS to establish equality between public sidecar and encrypted plaintext. Honest first-party senders duplicate the same signed event; endpoints validate the protected event and its capability context. The implementation does **not** provide an end-to-end consensus certificate proving that every peer applied the same sidecar/event pair. An authorized malicious endpoint can equivocate or disrupt progress; bounded models assume honest participating endpoint logic. Do not infer Byzantine agreement from signatures or relay acceptance.

F.2 Succession state transitions

State / input	Guard	Persisted outcome / next step
Host connected; policy update	Host signature, exact current admission order, valid ordered application	Retain signed policy and outer frame message ID
Host disconnects	Owning socket closes	Preserve authority during 120-second reconnect grace
Grace expires	Current host still absent	Fence old identity; stop its resume; cancel conflicting pending work
Offer candidate	Current policy; first connected active eligible unattempted device	Persist attempted candidate and 30-second authorization; backlog delivery
Authorization delivered	Exact policy/capability/offer binding	Candidate stores before delivery ACK; show catch-or-duck choice, not promotion
Candidate explicitly accepts	Valid signed response, target/socket/grant/order, current policy, unexpired capability	Commit new authority and consume capability **before** delivery; invalidate old policy; retire old host; retain exact removal marker
Candidate explicitly refuses	Same exact authorization checks	Consume that offer; keep refuser admitted; offer next eligible member
Offer expires/unanswered	Current offer reaches expiry	Next eligible unattempted member; never implicit acceptance
Exhaustion	No eligible authorized candidate	Retire room and close live connections: fort collapses
Successor cleans up	Exact former-host admission/Remove binding	Persist Remove, accepted result, retirement control and lifecycle completion
Successor refreshes policy	New host, reconciled membership, cleanup complete	Publish policy under the new host's own signing authority

Authorization IDs are deterministic per policy/candidate:

```
base64url(SHA256(UTF8("Pillowfort host succession v4\0" +
 roomInstance + "\0" + policyEventId + "\0" + deviceId))[0:16])
```

The candidate has no authority merely because its position was selected. Host software automates policy maintenance and cryptographic validation, not the recipient's decision.

F.3 Successor crash contract

Crash boundary	Durable state	Returns within its reconnect grace	Never returns before grace expires
Relay switch committed; result not received/ACKed	New relay host; exact accepted result; candidate outbox; former-host removal marker; old policy invalid	Device resume reconciles exact result and retained cleanup, then publishes own policy	No new policy exists: room retires rather than reuse predecessor delegation
Acceptance reconciled; removal unstarted	New host and exact historical removal obligation	Resume, process marker, complete exact Remove and lifecycle notification	Same safe termination; uninterrupted availability is not promised
Removal accepted/completed; new policy not relayed	New host, removal outcome/control; possibly durable unsent policy ciphertext	Resume reconciles old socket-bound grant and retries/recreates the policy under a fresh valid grant	Room retires if no new policy was accepted
New policy accepted	New host's signed policy	Resume normally	A new consent-based offer may proceed under that policy

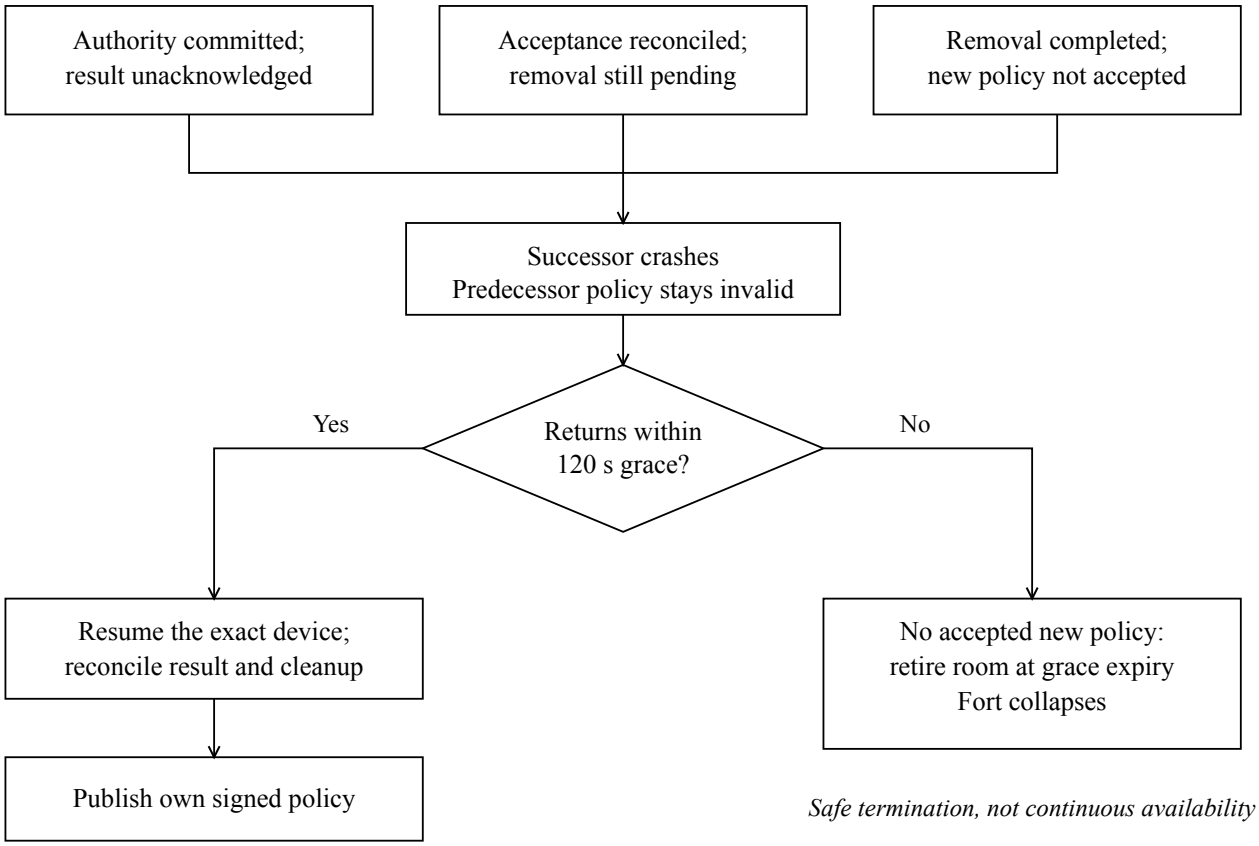


Figure 7. Successor loss before a new policy is accepted. The accepted authority switch is not reversed. Recovery uses the same durable identity and exact cleanup obligation; permanent loss without a current successor delegation terminates the room.

A dishonest relay can induce an authorized but unnecessary succession by falsely representing that a connected host is unavailable. It cannot forge candidate acceptance, but acceptance is not cryptographic evidence that replacement was necessary. This consequence follows directly from making the relay the connectivity/deadline oracle.

G. Executed model scope and implementation mapping

The models implement the existing plan's stages: M1 delivery; M2 admission/identity; M3 resume; M4 succession. `Coordination.tla` composes them in that order with admission, resume and host-availability send barriers. Separate state mutations represent persistence and network effects. Cryptography is an abstract authenticated capability; OpenMLS, IndexedDB and the transport are not formally verified by these models.

Standalone models explore their finite reachable graphs with up to two operation/KeyPackage tokens, backlog capacity two, three host/member identities, and before/after deadline regions. Composed profiles check prefixes of 12 steps, plus a 16-step configuration with backlog capacity two. These are **safety** checks; deadlock, fairness, real-time liveness, crash recovery of an unavailable storage device, and unbounded executions are excluded. The composed resume abstraction is an interlock, not a model of every repeated socket incarnation or all concrete host-view combinations.

The runner also weakens seven safeguards individually: persist-before-send, grant connection binding, host preview readiness, single-use KeyPackages, snapshot reconciliation, transfer target binding, and membership send interlocks. Every weakened model must produce an invariant counterexample. These traces demonstrate sensitivity of the abstraction. The host-preview guard corresponds to a separately reproduced and fixed implementation race; the other mutations are not demonstrated implementation vulnerabilities.

Plan property / failure	Concrete evidence	Limit
M1.1, M1.2, M1.3, M1.5	M1 model; engine persistence/restart/CAS test; relay fingerprint and cancellation tests	Two bounded operations, not unlimited replay retention
M1.4	M1 model and relay exact-prefix ACK tests	Resume uses its different whole-replay terminator
M2.1–M2.5	M2 model; every partial-admission phase expiry/cancel regression; ambiguous-auth browser engine tests	Signatures abstracted; bootstrap redaction checked concretely, not proved
M3.1–M3.5	M3 model; stale sentinel, historical marker, final-close and snapshot tests	Model does not establish global transcript agreement
M4.1–M4.8	M4/composed models; six successor crash-boundary regressions; actual browser boundary interception	Availability requires a willing recoverable coordinator or current new policy
Invalid peer membership transition	Engine commit-validation/rollback tests and controller fail-close path	No epoch merge or Byzantine agreement claim

H. Reproduction and measurement interpretation

```
git clone https://github.com/slee1996/pillowfort.git
cd pillowfort
git checkout --detach 10fe8f8b490c9e890d5a642bc25568e985c2fe8a
npm ci
npm --prefix client ci
npx playwright install chromium
npm run check
bun test test/secure-relay-v4.test.ts --test-name-pattern 'M4 successor crash boundaries'
curl --fail --location https://github.com/tlaplus/tlaplus/releases/download/v1.7.4/tla2tools.jar --output /tmp/tla2tools-1.7.4.jar
python3 scripts/check-protocol-models.py --jar /tmp/tla2tools-1.7.4.jar --output /tmp/pillowfort-models
bun scripts/protocol-browser-evidence.ts --mode benchmark --output /tmp/pillowfort-benchmarks
bun scripts/protocol-browser-evidence.ts --mode faults --output /tmp/pillowfort-browser-faults
```

Use Node/Bun versions required by the checked lockfiles, Java 21 or compatible, the pinned TLC JAR hash enforced by the runner, and the Chromium installed for pinned Playwright 1.65.0. Models, raw logs, exact generated configurations, sample arrays and platform data are included in the public evidence artifact. The browser script uses temporary local rooms, actual host approval and explicit candidate acceptance; it neither disables production rate limits nor shortens the 120-second grace.

Latency is measured from UI submission to observed application in a recipient DOM. It includes grant exchange, persistence, MLS, host preview, queueing and rendering. The slow-host profile adds 100 ms before delivering each host preview without changing cryptography. Each run sends one message at a time, then waits 500 ms after completion; there is no catch-up burst after a scheduler stall. Reported throughput is the **achieved closed-loop paced rate**, not a maximum-capacity or stable open-loop two-messages/second claim. Measurements use one local machine and shared Chromium/hardware; they are not WAN, Cloudflare or mobile results.

Quantiles use nearest-rank selection: sort the 20 samples and select index `ceil(p*n)-1`. The raw report's `medianMs` field is that p50 estimate (the lower central observation for an even sample count), not the average of the central pair. Retain all sample values when comparing estimators or reproducing the table.

I. Related designs

Design	Relevant mechanism	Distinction from Pillowfort
MLS architecture, RFC 9750 §§5.2–5.3, 6.4–6.6	Delivery-service consistency, application access control, invalid commits, state-loss recovery	These responsibilities are already recognized by MLS; Pillowfort specifies a particular room coordinator and persist-before-effect composition
Matrix Megolm, specification v1.19	Per-sender outbound ratchets, session-key distribution, application replay tracking	Different cryptographic/state organization; not an MLS epoch or Pillowfort coordinator benchmark baseline
Signal private group system	Server-canonicalized encrypted group state with privacy-preserving authorization proofs	Different metadata/privacy goals; Pillowfort's relay observes device/membership coordination metadata and uses explicit admitted-host authority

Sources: RFC 9750, Megolm v1.19, Signal private group design. This is a design comparison, not a claim that these systems were experimentally benchmarked or that Pillowfort improves their cryptography.